Introduction

The KiCad Team

Table of Contents

Welcome 2
Installing and Upgrading KiCad 4
  Migrating from Previous Versions 4
KiCad Workflow 5
  Basic Terminology 5
  KiCad Components 6
  User Interface 6
Further Reading 8

Copyright

This document is Copyright The KiCad Documentation Contributors. You may distribute it and/or modify it under the terms of either the GNU General Public License (http://www.gnu.org/licenses/gpl.html), version 3 or later, or the Creative Commons Attribution License (http://creativecommons.org/licenses/by/3.0/), version 3.0 or later.

All trademarks within this guide belong to their legitimate owners.

Contributors

Jon Evans, Graham Keeth

Feedback

The KiCad project welcomes feedback, bug reports, and suggestions related to the software or its documentation. For more information on how to submit feedback or report an issue, please see the instructions at https://www.kicad.org/help/report-an-issue/

Software and Documentation Version

This user manual is based on KiCad 10.0.8. Functionality and appearance may be different in other versions of KiCad.

Documentation revision: `fbe3d967`.

Welcome

KiCad is a free and open-source electronics design automation (EDA) suite. It features schematic capture, integrated circuit simulation, printed circuit board (PCB) layout, 3D rendering, and plotting/data export to numerous formats. KiCad also includes a high-quality component library featuring thousands of symbols, footprints, and 3D models. KiCad has minimal system requirements and runs on Linux, Windows, and macOS.

KiCad 10.0 is the most recent major release. It includes hundreds of new features and bug fixes. Some of the most notable new features include:

- Toolbars can be customized in all editors.
- A lasso selection mode has been added to the schematic and PCB editors.
- You can now import designs from Allegro, PADS, and gEDA/Lepton EDA.
- Design variants have been added, allowing you to define multiple versions of a single project that share a schematic and layout but have different properties (for example, different part numbers or certain parts that are not placed).
- Symbols and footprints can now have pins that are jumpered. Jumpered pins are considered to be internally connected inside the mounted component.
- Support for groups has been added to the Schematic Editor, matching the grouping features in the PCB Editor.
- You can now import and export pin definitions with CSV files in the Symbol Editor's Symbol Pins Table.
- Power symbols (VCC, GND, etc.) can now optionally be marked as *local* power symbols, which do not make connections between schematic sheets.
- Symbols can now have multiple alternate body styles. In previous versions of KiCad, symbols were limited to a single "De Morgan equivalent" alternate body style.
- The track length tuning system was overhauled. This includes improvements to length calculations to make the router more consistent with DRC, support for time-domain tuning constraints instead of simple length constraints, and the addition of Tuning Profiles, which let you define per-layer length tuning parameters.
- The Design Block feature has been extended to the PCB Editor, letting you maintain a library of schematic and layout fragments.
- Footprints can now contain graphics, keepouts, and other objects on inner layers, instead of just outer layers as in previous versions.
- An unconstrained pin/gate swap feature has been added to let you forward- and back-annotate pin and gate (unit) changes between the PCB and Schematic Editors.
- Custom DRC rules can now be edited in the Graphical DRC Rule Editor. The graphical editor is fully compatible with the existing textual custom rule system.
- The PCB and Footprint Editors can now generate and edit barcode objects, with support for several types of barcodes.

The concept of a "point" object was added to the PCB and Footprint Editors. Points are helpful for snapping and locating objects, but do not affect fabrication outputs.

- Significant improvements have been made to the Symbol, Footprint, and 3D Model libraries.

A full listing of new features and changes in KiCad 10.0 can be found here.

Installing and Upgrading KiCad

KiCad maintains compatibility and support with the maintained versions of Microsoft Windows, Apple macOS, and a number of Linux distributions. Some platforms have specific installation or upgrade instructions. Always check https://www.kicad.org/download/ for the latest release information and instructions for your platform.

KiCad may compile and run on platforms that are not officially supported. The KiCad development team makes no guarantees that KiCad will continue to work on these platforms in the future. See https://www.kicad.org/help/system-requirements/ for more details on supported platforms and hardware requirements.

KiCad uses a "major.minor.point" release version format. Major releases bring new features and other significant changes to the code. Minor releases are relatively rare and typically bring bug fixes that are too complicated for a point release. Point releases contain only bugfixes. Users are encouraged to update to the latest point release for their current major.minor version promptly, as these releases will not break file compatibility. Major releases almost always come with changes to file formats. KiCad is in general always backwards compatible with files created by older versions, but not forwards compatible: Once files are edited and saved by a new major version, these files will not be openable by the previous major version.

Migrating from Previous Versions

In general, to migrate a design to a new version of KiCad, simply open the project with the new version, then open the schematic and PCB and save each file. More details about specific issues that may come up when migrating designs is covered in the Schematic Editor and PCB Editor chapters of the manual.

NOTE	Make sure to save a backup of your design before opening it with a new version of KiCad. Once saved in a new major version of KiCad, designs can no longer be opened by previous major versions.

The symbol library format changed in KiCad 6.0. To continue editing symbol libraries made with a previous version of KiCad, these libraries need to be migrated to the new format. For details on this process, see the Schematic Editor chapter of the manual. Symbol libraries that have not been migrated can still be opened and used in read-only mode.

KiCad Workflow

This section presents a high-level overview of the typical KiCad workflow. Note that KiCad is a flexible software system, and there are other ways of working that are not described here. For more information about each of the steps described in this section, please see the later chapters in this manual.

NOTE	A number of tutorials and guided lessons in using KiCad have been created by community members. These resources can be a good way to learn KiCad for some new users. See the Further Reading section at the end of this chapter for more information.

Basic Terminology

KiCad uses a number of terms that are fairly standard in the area of electronics design automation (EDA) software, and some that are more specific to KiCad. This section lists some of the most common terms used throughout KiCad's documentation and user interface. Other terms that are more specific to a certain part of the KiCad workflow are defined later in this manual.

A **schematic** is a collection of one or more pages (sheets) of circuit schematic drawings. Each KiCad schematic file represents a single sheet.

A **hierarchical schematic** is a schematic consisting of multiple pages nested inside each other. KiCad supports hierarchical schematics, but there must be a single **root sheet** at the top of the hierarchy. Sheets within a hierarchy (other than the root sheet) may be used more than once, for example to create repeated copies of a subcircuit.

A **symbol** is a circuit element that can be placed on a schematic. Symbols can represent physical electrical components, such as a resistor or microcontroller, or non-physical concepts such as a power or ground rail. Symbols have **pins** which serve as the connection points that can be wired to each other in a schematic. For physical components, each pin corresponds to a distinct physical connection on the component (for example, a resistor symbol will have two pins, one for each terminal of the resistor). Symbols are stored in **symbol libraries** so they can be used in many schematics.

A **netlist** is a representation of a schematic that is used to convey information to another program. There are many netlist formats used by various EDA programs, and KiCad has its own netlist format that is used internally to pass information back and forth between the schematic and PCB editors. The netlist contains (among other things) all the information about which pins connect to each other, and what name should be given to each **net**, or set of connected pins. Netlists can be written to a **netlist file**, but in modern versions of KiCad, this is not necessary as part of the normal workflow.

A **printed circuit board**, or PCB, is a design document that represents the physical implementation of a schematic (or technically, a netlist). Each KiCad board file refers to a single PCB design. There is no official support for creating arrays or panels of PCBs within KiCad, although some community-created add-ons provide this functionality.

A **footprint** is a circuit element that can be placed on a PCB. Footprints often represent physical electrical components, but can also be used as a library of design elements (silkscreen logos, copper antennas and coils, etc.). Footprints can have **pads** which represent copper areas that are electrically-connected. The netlist will associate symbol pins with footprint pads.

A **worksheet** is a drawing template, typically containing a title block and frame, that is used as the template for schematic sheets and PCB drawings.

Plotting is the process of creating manufacturing outputs from a design. These outputs may include machine-readable formats such as Gerber files or pick-and-place listings, as well as human-readable formats such as PDF drawings.

Ngspice is a mixed-signal circuit simulator, originally based on Berkeley SPICE, that is integrated into KiCad's schematic editor. By using symbols with attached SPICE models, you can run circuit simulations on KiCad schematics and plot the results graphically.

KiCad Components

KiCad consists of a number of different software components, some of which are integrated together to facilitate the PCB design workflow, and some of which are standalone. In early versions of KiCad, there was very little integration between the software components. For example, the schematic editor (historically called Eeschema) and PCB editor (historically called PcbNew) were separate applications that had no direct link, and to create a PCB based on a schematic, users had to generate a netlist file in Eeschema and then read this netlist file in PcbNew. In modern versions of KiCad, the schematic and PCB editor are integrated into the KiCad project manager, and using netlist files is no longer required. Many tutorials still exist that refer to the old KiCad workflow of separate applications and netlist files, so be sure to check the version being used when looking at tutorials and other documentation.

The main KiCad components are usually started from the launcher buttons in the KiCad project manager window. These components include:

Component name	Description
Schematic Editor	Create and edit schematics; simulate circuits with SPICE; generate BOM files
Symbol Editor	Create and edit schematic symbols and manage symbol libraries
PCB Editor	Create and edit PCBs; export 2D and 3D files; generate fabrication output files
Footprint Editor	Create and edit PCB component footprints and manage footprint libraries
Gerber Viewer	Gerber and drill file viewer
Image Converter	Convert bitmap images to symbols or footprints
PCB Calculator	Calculator for components, track width, electrical spacing, color codes, etc.
Drawing Sheet Editor	Create and edit worksheet files

User Interface

KiCad has a number of user interface behaviors that are common to all the different editor windows. Some of these behaviors are described in more detail in later chapters of this manual.

Objects can be selected by clicking on them or by dragging a selection window around them. Dragging from left to right will result in a selection of any items that are completely within the window. Dragging from right to left will result in a selection of any items that touch the window. Pressing certain modifier keys while

clicking or dragging will change the selection behavior. These keys are platform-specific and are described in the Editing Options section of the Preferences dialog.

KiCad editors have the concept of a **tool** which can be thought of as a mode that the editor is in. The default tool is the selection tool, which means that clicking will select objects under the mouse cursor. There are also tools for placing new objects, inspecting existing objects, etc. The active tool is highlighted in the toolbar, and the name of the active tool is shown in the bottom right of the editor in the status bar. Pressing Esc always means "cancel" in KiCad: if a tool is in the middle of some action (for example, routing tracks), the first press of Esc will cancel that action. The next press of Esc will exit the tool complely, returning to the default selection tool. With the selection tool active, pressing Esc will clear the current selection, if one exists.

Further Reading

The latest version of this manual can be found in multiple languages at https://docs.kicad.org Manuals for previous versions of KiCad can also be found at that website.

The KiCad user community includes a number of forums and chat platforms that are operated independently from the KiCad development team but are fully endorsed as a great way to find help with problems, learn tips and tricks, and share examples of KiCad projects. A listing of community resources is available under the Community heading at https://www.kicad.org

Users interested in compiling KiCad from source and/or contributing to KiCad development should visit our developer documentation site at https://dev-docs.kicad.org for instructions, policies and guidelines, and technical information about the KiCad codebase.